MLOPs LAB

P25DS3P9



Name : _____

Register Number : _____

PG DEPARTMENT OF DATA SCIENCE
BISHOP HEBER COLLEGE (AUTONOMOUS)
(Nationally Re-accredited with A++ Grade by NAAC with a CGPA OF 3.69 out of 4)
(Recognized by UGC as College of Excellence", Affiliated to Bharathidasan University)
Tiruchirappalli - 620 017
October- 2026

PG DEPARTMENT OF DATA SCIENCE

BISHOP HEBER COLLEGE (AUTONOMOUS)

Tiruchirappalli- 620 017 Tamil Nadu

(Nationally Re-accredited with A++ Grade by NAAC with a CGPA OF 3.69 out of 4)

(Recognized by UGC as College of Excellence", Affiliated to Bharathidasan University)

BONAFIDE CERTIFICATE

Name : _____

Register No : _____

Class : _____

Course Code : **P25DS3P9**

Course Title : **MLOPs LAB**

Certified that this is the Bonafide Record of work done by me during **Odd**

Semester of **2026** submitted to the Practical Examination on _____

Staff In-Charge **Head of the Department**

Examiners

1._____

2._____

INDEX PAGE

S. No	Date	Title of the Program	Page.No	Sign
1		Setting up the MLOPs Environment & Basic Automation		
2		Introduction to Version Control for ML Projects		
3		Building Basic Data Pipelines		
4		Feature Engineering and Management		
5		Experiment Tracking with MLflow/Weights & Biases (Part 1)		
6		Experiment Tracking with MLflow/Weights & Biases (Part 2) & Model Versioning		
7		Containerization with Docker (Part 1)		
8		Containerization with Docker (Part 2) & Basic Deployment		
9		Introduction to Continuous Integration for ML		
10		Implementing Basic Continuous Delivery for ML		
11		Model Monitoring (Part 1) - Logging and Performance Metrics		
12		Model Monitoring (Part 2) - Drift Detection (Introduction) & Dashboard/Alerting		

1.Setting up the MLOPs Environment & Basic Automation

Aim:

To set up a basic MLOPs project environment, train a Decision Tree model using the Iris dataset, record the training process using logging, save the trained model, generate a training report, and load the model for prediction.

Procedure:

1. Import the required Python libraries.
2. Create the data, models, logs, and reports folders.
3. Configure logging to store training information.
4. Load the Iris dataset.
5. Divide the dataset into training and testing data.
6. Create and train a Decision Tree Classifier.
7. predict the test data and calculate accuracy.
8. Save the trained model using Joblib.
9. Generate a training report containing the date, algorithm, accuracy and status.
10. Load the saved model.
11. Provide a sample flower measurement and predict its class.

Code:

```
import os
import logging
import joblib
import pandas as pd
import pandas as pd
from datetime import datetime
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

folders = [
    "data",
    "models",
    "logs",
    "reports"
]

for folder in folders:
    os.makedirs(folder, exist_ok=True)

print("Folders Created")
```

```
logging.basicConfig(
    filename="logs/training.log",
    level=logging.INFO
)

logging.info("Training Started")

print("Training Started")


iris = load_iris()

X = iris.data
y = iris.target

logging.info("Dataset Loaded")


X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

logging.info("Dataset Split Completed")


model = DecisionTreeClassifier()

model.fit(X_train, y_train)

logging.info("Model Training Completed")


prediction = model.predict(X_test)


accuracy = accuracy_score(y_test, prediction)

print("Accuracy =", accuracy)

logging.info(f"Accuracy = {accuracy}")
```

```
joblib.dump(
    model,
    "models/iris_model.pk1")

print("Model Saved")


report = f"""
Training Date : {datetime.now()}

Algorithm : Decision Tree

Accuracy : {accuracy}

Status : Success
"""

with open("reports/report.txt", "w") as file:
    file.write(report)

print("Report Generated")


saved_model = joblib.load("models/iris_model.pk1")

print("Saved Model Loaded")


sample = [[5.1, 3.5, 1.4, 0.2]]

result = saved_model.predict(sample)

print("Prediction =", result)
```

Output:

Folders Created
Training Started
Accuracy = 1.0
Model Saved
Report Generated
Saved Model Loaded
Prediction = [0]

Result

Thus, the MLOPs program was successfully executed, and the expected output was obtained.

2.Introduction to Version Control for ML Projects

Aim:

To understand and implement version control for a machine learning project using Git by creating a repository, tracking files, committing changes, and maintaining different versions of the project.

Procedure:

1. Install Git on the computer.
2. Create a new folder for the machine learning project.
3. Open the terminal inside the project folder.
4. Initialize a Git repository using git init.
5. Create or add the required project files.
6. Check the status of files using git status.
7. Add the files to the staging area using git add.
8. Save the project version using git commit.
9. Modify the project files and commit the new changes.
10. View the commit history using git log.

Code:

1. Load Dataset

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

print(df.head())
print(df.shape)
```

2. Train/Test Split

```
from sklearn.model_selection import train_test_split

X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(    X, y, test_size=0.2,
random_state=42 )
print("Train size:", X_train.shape)
print("Test size:", X_test.shape)
```

3. Create Random Forest Model

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier( n_estimators=100, random_state=42 )
```

4. Train the Model

```
model.fit(X_train, y_train)
```

5. Calculate Accuracy

```
from sklearn.metrics import accuracy_score
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
```

6. Make Predictions

```
# Predict on a single test sample
sample = X_test.iloc[[0]]
prediction = model.predict(sample)
 print("Predicted class:", prediction)

# Predict on new custom data
new_data = [[5.1, 3.5, 1.4, 0.2]]
# must match feature order
print("Prediction for new data:", model.predict(new_data))
```

7. Version Control the Project with Git

Run these commands in the VS Code integrated terminal (Terminal > New Terminal):

```
# 1. Initialize Git repo
git init

# 2. Create a .gitignore file
echo "*.pyc __pycache__/
.ipynb_checkpoints/
venv/
 .env" > .gitignore

# 3. Stage all files
git add .

# 4. Commit with a meaningful message
git commit -m "Week 2: Random Forest model with train/test split"
```

5. Connect to a GitHub remote repository git remote add origin https://github.com/yourusername/your-repo.git

6. Push to GitHub
git branch -M main
git push -u origin main

Useful Follow-up Git Commands

Check current status
git status

View commit history
git log --oneline

Create and switch to a new branch (e.g. for tuning experiments)
git checkout -b experiment-tuning

Switch back to main branch git checkout main # Pull latest changes from GitHub
git pull origin main

Output:

Initialized empty Git repository in the project folder.

Files added successfully.
Changes committed successfully.

Commit history:
Initial commit - ML project files added
Second commit - ML model updated

Result:

Thus, the MLOPs program was successfully executed, and the expected output was obtained.

3. Building Basic Data Pipelines

Aim:

Build a basic ETL pipeline that reads a CSV file, cleans missing values, transforms data, and saves a new CSV.

Procedure:

1. Import the required Python libraries such as Pandas and Scikit-learn.
2. Load the dataset from a CSV file or a built-in dataset.
3. Display the dataset's first few records.
4. Check the dataset for missing values.
5. Handle missing values by removing or replacing them.
6. Separate the input features and target variable.
7. Apply preprocessing techniques such as feature scaling.
8. Divide the data into training and testing sets.
9. Create a pipeline containing preprocessing and a machine learning model.
10. Train the pipeline using the training data.
11. Predict the results for the testing data.
12. Evaluate the pipeline using an appropriate performance metric.
13. Display the model's final accuracy.

Code:

Step 1: Import Library

```
import pandas as pd
```

Step 2: students.csv

```
Name,Age,Marks
Arun,20,85
Priya,21,90
Rahul,,78
Meena,22,
Kumar,20,88
```

Step 3: Read CSV

```
df = pd.read_csv('students.csv')
```

Step 4: Check Missing Values

```
print(df.isnull().sum())
```

Step 5: Fill Missing Values

```
df['Age']=df['Age'].fillna(df['Age'].mean())
df['Marks']=df['Marks'].fillna(df['Marks'].mean())
```

Step 6: Add Grade

```
df['Grade']=df['Marks'].apply(lambda x:'Pass' if x>=50 else 'Fail')
```

Step 7: Sort

```
df=df.sort_values(by='Marks', ascending=False)
```

Step 8: Save

```
df.to_csv('cleaned_students.csv', index=False)
```

Complete Pipeline

```
import pandas as pd
df=pd.read_csv('students.csv')
df['Age']=df['Age'].fillna(df['Age'].mean())
df['Marks']=df['Marks'].fillna(df['Marks'].mean())
df['Grade']=df['Marks'].apply(lambda x:'Pass' if x>=50 else 'Fail')
df=df.sort_values(by='Marks',ascending=False)

df.to_csv('cleaned_students.csv',index=False)
print('Pipeline Completed Successfully!')
```

Output:

Missing Values

```
Name     0
Age      1
Marks    1
dtype: int64
```

Console Output

```
Pipeline Completed Successfully!
```

cleaned_students.csv

```
Name,Age,Marks,Grade
Priya,21.0,90.0,Pass
Kumar,20.0,88.0,Pass
Meena,22.0,85.25,Pass
Arun,20.0,85.0,Pass
Rahul,20.75,78.0,Pass
```

Result:

Thus, the MLOPs program was successfully executed, and the expected output was obtained.

4. Feature Engineering and Management

Aim:

To perform feature engineering and feature management on the California Housing dataset by creating meaningful derived features, handling missing values, and preparing the dataset for machine learning.

PROCEDURE

1. Import the required libraries.
2. Load the California Housing dataset using Scikit-learn.
3. Convert the dataset into a Pandas DataFrame.
4. Examine the original dataset.
5. Create new features from the existing columns.
6. Create RoomsPerHousehold.
7. Create BedroomsPerRoom.
8. Create PopulationPerHousehold.
9. Handle infinite values generated during feature creation.
10. Handle missing values using median imputation.
11. Save the feature-engineered dataset as a CSV file.
12. Display the original and final dataset information.

CODE

```
# Feature Engineering and Management
import pandas as pd
from sklearn.datasets
import fetch_california_housing
from sklearn.impute import SimpleImputer

# Load Dataset
housing = fetch_california_housing( as_frame=True )
df = housing.frame

print("Original Dataset")
print(df.head())
print("\nOriginal Shape:")
print(df.shape)

# Feature Engineering
df["RoomsPerHousehold"] = (df["AveRooms"] /  df["AveOccup"] )
df["BedroomsPerRoom"] = (df["AveBedrms"] /  df["AveRooms"] )
df["PopulationPerHousehold"] = ( df["Population"] / df["AveOccup"] )
```

```
# Replace Infinite Values
df.replace([float("inf"), float("-inf")],  pd.NA,     inplace=True )

# Handle Missing Values
numeric_columns = df.select_dtypes(include=["float64", "int64"] ).columns
imputer = SimpleImputer( strategy="median" )
df[numeric_columns] = imputer.fit_transform( df[numeric_columns] )

# Display Result
print("\nNew Features:")
print("RoomsPerHousehold") print("BedroomsPerRoom")
print("PopulationPerHousehold")
print("\nFinal Shape:")
print(df.shape)

# Save Dataset
df.to_csv( "feature_engineered_housing.csv", index=False )
print("\nDataset saved successfully.")
```

Output:

Original Dataset

```
MedInc  HouseAge  AveRooms  AveBedrms ...
0   8.325      41.0     6.984      1.024 1
8.301      21.0     6.238      0.972 ...
```

Original Shape:

(20640, 9)

New Features:

RoomsPerHousehold
BedroomsPerRoom
PopulationPerHousehold

Final Shape:

(20640, 12)

Dataset saved successfully.

Result:

Thus, the MLOPs program was successfully executed, and the expected output was obtained.

5. Experiment Tracking with MLflow/Weights & Biases (Part 1)

Aim:

To implement experiment tracking using MLflow by training multiple machine learning models on the Iris dataset and recording model parameters and evaluation metrics.

PROCEDURE

1. Import the required libraries.
2. Load the Iris dataset.
3. Separate features and target values.
4. Divide the dataset into training and testing sets.
5. Create Linear/Logistic Regression, Decision Tree and Random Forest models.
6. Train each model.
7. Calculate accuracy for each model.
8. Create an MLflow experiment.
9. Log model parameters using MLflow.
10. Log accuracy metrics using MLflow.
11. Compare the performance of the models.
12. Display the experiment results.

CODE

```
# Experiment Tracking using MLflow
import mlflow
import mlflow.sklearn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load Dataset
iris = load_iris()
X = iris.data
y = iris.target

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.20, random_state=42 )
```

```
# MLflow Setup
 mlflow.set_experiment( "Iris_Experiment_Tracking" )

# Models
 models = {"Logistic Regression":LogisticRegression(max_iter=200),
"Decision Tree": DecisionTreeClassifier(random_state=42),
"Random Forest": RandomForestClassifier( n_estimators=100, random_state=42)
}
 # Train Models
 for name, model in models.items():
     with mlflow.start_run(run_name=name):
         model.fit(X_train, y_train)
         predictions = model.predict(X_test)
         accuracy = accuracy_score(y_test, predictions)

# Log Experiment
mlflow.log_param( "model", name )
mlflow.log_metric("accuracy", accuracy)
mlflow.sklearn.log_model("model", model )
print("\nModel:", name)
print("Accuracy:", round(accuracy, 4))
print("\nExperiment Tracking Completed.")
```

OUTPUT

```
Model: Logistic Regression
Accuracy: 1.0
Model: Decision Tree
Accuracy: 1.0
Model: Random Forest
Accuracy: 1.0
Experiment Tracking Completed.
The experiments can be viewed using: mlflow ui
Then open: http://localhost:5000
```

Result:

Thus, the MLOPs program was successfully executed, and the expected output was obtained.

6. Experiment Tracking with MLflow/Weights & Biases (Part 2) & Model Versioning

Aim:

To implement experiment tracking and model versioning using MLflow by training multiple machine learning models on the Iris dataset, recording their parameters and accuracy, and registering different model versions for comparison and management.

Procedure:

1. Import MLflow, Scikit-learn models, dataset, and evaluation libraries.
2. Load the Iris dataset using load_iris().
3. Separate the dataset into input features X and target labels y.
4. Split the data into training and testing sets using train_test_split().
5. Create an MLflow experiment using mlflow.set_experiment().
6. Define Logistic Regression, Decision Tree, and Random Forest classifiers.
7. Start a separate MLflow run for each machine learning model.
8. Train each model and generate predictions using the test dataset.
9. Calculate accuracy and log the model parameters, metrics, and trained model in MLflow.
10. Register the trained models in the MLflow Model Registry to maintain model versions and compare experiment results.

Code:

```
pip install mlflow
import mlflow
import mlflow.sklearn

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

1. Load Dataset

```
iris = load_iris()
X = iris.data
y = iris.target
```

2. Train-Test Split

```
X_train, X_test, y_train, y_test
= train_test_split(X,y,test_size=0.20,random_state=42)
```

3. MLflow Experiment Setup

```
mlflow.set_experiment("Iris_Experiment_Tracking")
```

4. Define Models

```
models = {
"Logistic Regression":LogisticRegression(max_iter=200),
"Decision Tree":DecisionTreeClassifier(random_state=42),
"Random Forest":RandomForestClassifier(n_estimators=100,random_state=42)
}
```

Registered Model Name

```
registered_model_name = "Iris_Classifier"
```

5. Train and Track Models

```
for name, model in models.items():
with mlflow.start_run(run_name=name):
```

Train

```
model.fit(X_train,y_train)
```

Prediction

```
predictions = model.predict(X_test)
```

Accuracy

```
accuracy = accuracy_score(y_test,predictions)
```

Log Parameters

```
mlflow.log_param("model_name",name)
```

Log Metrics

```
mlflow.log_metric("accuracy",accuracy)
```

Log + Register Model

```
mlflow.sklearn.log_model(
sk_model=model,
name="model",
registered_model_name=registered_model_name,
input_example=X_train[:5]
)
print("\nModel:", name)
print("Accuracy:",round(accuracy, 4))
print("\nExperiment Tracking and "
"Model Versioning Completed."
)
```

Output:

Downloading artifacts: 100% 7/7 [00:00<00:00, 263.44it/s]

Registered model 'Iris_Classifier' already exists. Creating a new version of this model...
Created version '4' of model 'Iris_Classifier'.

Model: Logistic Regression
Accuracy: 1.0

Downloading artifacts: 100% 7/7 [00:00<00:00, 324.27it/s]

Registered model 'Iris_Classifier' already exists. Creating a new version of this model...
Created version '5' of model 'Iris_Classifier'.

Model: Decision Tree
Accuracy: 1.0

Downloading artifacts: 100% 7/7 [00:00<00:00, 319.25it/s]

Model: Random Forest
Accuracy: 1.0

Experiment Tracking and Model Versioning Completed.

Registered model 'Iris_Classifier' already exists. Creating a new version of this model...
Created version '6' of model 'Iris_Classifier'.

Result:

Thus, the MLOPs program was successfully executed, and the expected output was obtained.

7. Containerization with Docker (Part 1)

Aim:

To containerize a machine learning application using Docker, ensuring reproducible environments,dependency management, and automated model training and prediction execution inside an isolated container.

Procedure

1. Prepare the machine learning script (train.py) to load the Iris dataset, train a Random Forest Classifier, and serialize the model using joblib.
2. Create an inference script (predict.py) to load the trained model and perform predictions on sample input data.
3. Define project dependencies in requirements.txt (including scikit-learn, joblib, Flask, and gunicorn).
4. Write a Dockerfile specifying the base Python image, working directory, dependency installation, code copying, and execution commands.
5. Build and run the Docker container to execute training and prediction steps in an isolated environment.

Code:

File Name: train.py

Python

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
import joblib

iris = load_iris()
X = iris.data
y = iris.target
model = RandomForestClassifier()
model.fit(X, y)

joblib.dump(model, "model.pkl")
print("Model saved successfully.")
```

Output:

Model saved successfully.

File Name: predict.py

```
import joblib

# Load model
model = joblib.load("model.pkl")

# Sample input
sample = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(sample)
print("Prediction:", prediction[0])
```

Output:

Prediction: 0

File Name: requirements.txt

```
Flask
scikit-learn
joblib
gunicorn
```

Output:

Dependencies specified successfully.

File Name: Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python train.py
EXPOSE 5000
CMD ["python", "predict.py"]
```

Output:

Container built and executed successfully.

Result:

Thus, the MLOps program was successfully executed, and the expected output was obtained.

8. Containerization with Docker (Part 2) & Basic Deployment

Aim:

To deploy the containerized machine learning model as a web service using Flask, configure a production-ready Dockerfile, and perform basic deployment and validation aligned with Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure).

Procedure:

1. Develop a Flask application (app.py) to load the serialized model (model.pkl) and expose REST API endpoints (/ for status check and /predict for real-time inference).
2. Update the Dockerfile to set up the working directory, copy requirements, install dependencies, copy application source code, and define startup commands.
3. Build the Docker container image and run it in detached mode while mapping container port 5000 to the host machine.
4. Verify API functionality by sending HTTP GET and POST requests to the containerized endpoints.
5. Validate service resilience and infrastructure reliability supporting robust industrial innovation

Code:

File Name: app.py

```
from flask import Flask, request, jsonify
import joblib
import numpy as np

app = Flask(__name__)
model = joblib.load("model.pkl")
@app.route("/", methods=["GET"])
def home():
    return jsonify({"status": "running", "message": "ML Model API is live!"})

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json(force=True)
    features = np.array(data["features"]).reshape(1, -1)
    prediction = model.predict(features)
    return jsonify({"prediction": int(prediction[0])})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Output:

Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)

File Name: Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Output:

Successfully built container image 'ml-api:latest' and bound to port 5000.

Result:

Thus, the MLOPs program was successfully executed, and the expected output was obtained.

9. Introduction to Continuous Integration for ML

Aim:

To understand and implement Continuous Integration (CI) principles for Machine Learning by developing an automated testing pipeline that validates model existence, prediction integrity, and accuracy thresholds using a Random Forest Classifier.

Procedure:

1. Import necessary libraries including Pandas, NumPy, and Scikit-Learn.
2. Load the Iris dataset and organize it into a feature matrix (X) and target array (y).
3. Split the dataset into training (80%) and testing (20%) sets using stratified sampling to ensure proportional class distribution.
4. Instantiate and train a RandomForestClassifier model using the training data.
5. Generate model predictions utilizing the unseen testing dataset.
6. Calculate the operational accuracy of the model to measure its performance.
7. Execute a primary Continuous Integration block to verify if the model meets a strict 90% accuracy threshold.
8. Perform subsequent assert-based diagnostic tests to validate model instantiation, dimensional integrity, and expected categorical prediction values.
9. Determine and output the final pipeline status (PASS/FAIL) based on the comprehensive test suite results.

Code:

1. IMPORT LIBRARIES

```
import os
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
print("WEEK 9 - CONTINUOUS INTEGRATION FOR ML")
```

2. LOAD DATASET

```
iris = load_iris()
X = pd.DataFrame( iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)

print("\n1. DATASET LOADED")
print("Dataset shape:", X.shape)
display(X.head())
```

3. SPLIT DATA

```
X_train, X_test, y_train, y_test = train_test_split(X,
y,test_size=0.2,random_state=42,stratify=y)

print("\n2. DATASET SPLIT")
print("-" * 60)
print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))
```

4. TRAIN ML MODEL

```
model = RandomForestClassifier(n_estimators=100,random_state=42)
model.fit(X_train,y_train)
print("\n3. MODEL TRAINED")
print("Algorithm: Random Forest")
```

5. MAKE PREDICTIONS

```
predictions = model.predict(X_test)
print("\n4. PREDICTIONS COMPLETED")
print("Predictions:")
print(predictions)
```

6. CALCULATE MODEL ACCURACY

```
accuracy = accuracy_score(y_test,predictions)
print("\n5. MODEL PERFORMANCE")
print("Accuracy:",round(accuracy, 4))
```

7. CONTINUOUS INTEGRATION TEST

```
minimum_accuracy = 0.90
print("\n6. CI TEST")
print("Required Accuracy:",minimum_accuracy)
print("Actual Accuracy :",round(accuracy, 4))
if accuracy >= minimum_accuracy:
   print("\n CI TEST PASSED")
   print("Model accuracy satisfies ""the required threshold.")
else:
   print("\n CI TEST FAILED")
print("Model accuracy is below ""the required threshold.")
```

8. ADDITIONAL TESTS

```
print("\n7. ADDITIONAL MODEL TESTS")
print("-" * 60)
```

Test 1: Model exists
assert model is not None
print(" Model existence test passed")

Test 2: Prediction count
assert len(predictions) == len(y_test)
print(" Prediction count test passed")

Test 3: Accuracy threshold
assert accuracy >= minimum_accuracy
print(" Accuracy threshold test passed")

Test 4: Prediction values
assert all(
prediction in [0, 1, 2]
for prediction in predictions)
print(" Prediction value test passed")

9. FINAL CI STATUS
print("\nCONTINUOUS INTEGRATION RESULT")
if accuracy >= minimum_accuracy:
    print("STATUS: PASS")
    print("All ML tests completed successfully.")
else:
    print("STATUS: FAIL")

print("\nWeek 9 exercise completed successfully!")

Output:

1. DATASET LOADED
Dataset shape: (150, 4)
sepal length (cm) sepal width (cm) petal length (cm) petal width (cm)
0 5.1 3.5 1.4 0.2
1 4.9 3.0 1.4 0.2
2 4.7 3.2 1.3 0.2
3 4.6 3.1 1.5 0.2
4 5.0 3.6 1.4 0.2

2. DATASET SPLIT

Training samples: 120
Testing samples : 30

3. MODEL TRAINED
Algorithm: Random Forest

4. PREDICTIONS COMPLETED
Predictions:[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 1 0 2 1 1 2 2 1 0 2 0]

5. MODEL PERFORMANCE
Accuracy: 0.9

6. CI TEST
Required Accuracy: 0.9
Actual Accuracy : 0.9
CI TEST PASSED
Model accuracy satisfies the required threshold.

7. ADDITIONAL MODEL TESTS

Model existence test passed
Prediction count test passed
Accuracy threshold test passed
Prediction value test passed
CONTINUOUS INTEGRATION RESULT
STATUS: PASS
All ML tests completed successfully.
Week 9 exercise completed successfully!

Result:

Thus, the MLOPs program was successfully executed, and the expected output was obtained.

10.Implementing Basic Continuous Delivery for ML

Aim:

To implement a basic Continuous Delivery (CD) pipeline in Machine Learning by automating model evaluation, artifact generation, metadata logging, and deployment verification against predefined performance thresholds.

Procedure:

1. Import foundational libraries for data manipulation (Pandas), modeling (Scikit-Learn), and model serialization (Joblib).
2. Programmatically construct a dedicated directory (model_artifacts) to store deployable model assets securely.
3. Load the Iris dataset and partition it into structured training and testing sets utilizing stratified sampling.
4. Train a robust RandomForestClassifier algorithm on the prepared training subset.
5. Evaluate model predictions employing both an overarching accuracy score and a granular classification_report.
6. Enforce a Continuous Delivery quality gate requiring a minimum accuracy benchmark of 90%.
7. Conditioned on passing the threshold, serialize and save the trained model object as a .pkl file in the artifact directory.
8. Write a corresponding metadata text file (model_info.txt) to log operational attributes such as algorithm type, dataset origin, actual accuracy, and delivery status.
9. Verify successful delivery by reloading the serialized model from disk and executing a sample prediction to confirm its structural integrity.

Code:

1. IMPORT LIBRARIES

```
import os
import joblib
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, classification_report)

print("=" * 60)
print("WEEK 10 - CONTINUOUS DELIVERY FOR ML")
print("=" * 60)
```

2. CREATE MODEL DIRECTORY

```
MODEL_FOLDER = "model_artifacts"
os.makedirs(MODEL_FOLDER, exist_ok=True)
print("\n1. MODEL DIRECTORY CREATED")
print("-" * 60)
print("Folder:", MODEL_FOLDER)
```

3. LOAD DATASET

```
iris = load_iris()

X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)
print("\n2. DATASET LOADED")
print("-" * 60)
print("Dataset shape:", X.shape)
```

4. SPLIT DATA

```
X_train, X_test, y_train, y_test =
train_test_split(X,y,test_size=0.20,random_state=42,stratify=y)
print("\n3. DATASET SPLIT")
print("-" * 60)
print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))
```

5. TRAIN MODEL

```
model = RandomForestClassifier(n_estimators=100,random_state=42)
model.fit(X_train,y_train)
print("\n4. MODEL TRAINING")
print("-" * 60)
print("Random Forest model trained successfully.")
```

6. PREDICTION

```
predictions = model.predict(X_test)
```

7. EVALUATE MODEL

```
 accuracy = accuracy_score(y_test, predictions)
print("\n5. MODEL EVALUATION")
print("-" * 60)
print("Accuracy:",round(accuracy, 4))
print("\nClassification Report:")
print(classification_report(y_test, predictions, target_names=iris.target_names))
```

8. DELIVERY THRESHOLD

```
minimum_accuracy = 0.90
print("\n6. DELIVERY EVALUATION")
print("-" * 60)
print("Minimum required accuracy:",minimum_accuracy)
print("Achieved accuracy:",round(accuracy, 4))
```

9. CONTINUOUS DELIVERY

```
if accuracy >= minimum_accuracy:
    print("\n MODEL APPROVED FOR DELIVERY")
#Save model
model_path = os.path.join(MODEL_FOLDER,"iris_model.pkl")
joblib.dump(model,model_path)
print("Model saved successfully:")
print(model_path)
else:
print("\n MODEL REJECTED")
print("Model was not delivered because "
"accuracy is below the threshold."
)
```

10. SAVE MODEL INFORMATION

```
if accuracy >= minimum_accuracy:
model_info = {
"model": "Random Forest",
"dataset": "Iris",
"accuracy": float(accuracy),
"threshold": minimum_accuracy,
"status": "APPROVED"
}
info_path = os.path.join(
MODEL_FOLDER,
"model_info.txt"
)
with open(info_path,"w") as file:
for key, value in model_info.items():
   file.write(f"{key}: {value}\n")
   print("Model information saved:")
print(info_path)
```

11. VERIFY DELIVERED MODEL

```
if accuracy >= minimum_accuracy:
print("\n7. DELIVERY VERIFICATION")
print("-" * 60)
```

```
if os.path.exists(model_path):
    print("Model artifact exists")
    loaded_model = joblib.load(model_path)
    print("Model loaded successfully")
    test_prediction = loaded_model.predict(X_test.iloc[:5])
    print("Test predictions:",test_prediction)
else:
print(" Model artifact not found.")

12. FINAL RESULT
print("\n")
print("=" * 60)
print("CONTINUOUS DELIVERY RESULT")
print("=" * 60)
if accuracy >= minimum_accuracy:
    print("STATUS: DELIVERED")
    print("The trained ML model passed evaluation "
            "and was saved as a deployable artifact.")
else:
    print("STATUS: REJECTED")

print("=" * 60)
print("\nWeek 10 exercise completed successfully!")
```

Output:

1. MODEL DIRECTORY CREATED
Folder: model_artifacts

2. DATASET LOADED
Dataset shape: (150, 4)

3. DATASET SPLIT
Training samples: 120
Testing samples : 30

4. MODEL TRAINING
Random Forest model trained successfully.

5. MODEL EVALUATION
Accuracy: 0.9

Classification Report:
precision recall f1-score support
setosa 1.00 1.00 1.00 10
versicolor 0.82 0.90 0.86 10
virginica 0.89 0.80 0.84 10
accuracy 0.90 30
macro avg 0.90 0.90 0.90 30
weighted avg 0.90 0.90 0.90 30

6. DELIVERY EVALUATION
Minimum required accuracy: 0.9
Achieved accuracy: 0.9
MODEL APPROVED FOR DELIVERY
Model saved successfully:
model_artifacts/iris_model.pkl
Model information saved:
model_artifacts/model_info.txt

7. DELIVERY VERIFICATION
Model artifact exists
Model loaded successfully
Test predictions: [0 2 1 1 0]

CONTINUOUS DELIVERY RESULT
STATUS: DELIVERED
The trained ML model passed evaluation and was saved as a deployable artifact.
Week 10 exercise completed successfully!

Result:

Thus, the MLOPs program was successfully executed, and the expected output was obtained.

11.Model Monitoring (Part 1) - Logging and Performance Metrics

Aim:

To implement comprehensive model monitoring by generating performance metrics, logging operational data, and producing automated alerts and reports for a deployed Machine Learning model.

Procedure:

1. Import foundational modules for data manipulation, metric calculation, logging, and system operations.
2. Create a dedicated directory (week11_monitoring) to securely store generated log files, CSVs, and diagnostic reports.
3. Configure Python's logging system to record timestamped system events to model_monitoring.log.
4. Load the deployed model artifact (iris_model.pkl) verified during the Continuous Delivery phase (Week 10).
5. Reconstruct the test dataset using the original data schemas and stratified sampling methods.
6. Execute the model to generate predictions on the test set and calculate primary statistical metrics: Accuracy, Precision, Recall, and F1-Score.
7. Diagnose model behavior using granular tools such as a Confusion Matrix and a Classification Report.
8. Compile the evaluated metrics into a Pandas DataFrame, appending a timestamp, and export this as a historical tracking file (performance_metrics.csv).
9. Implement an operational threshold alert that triggers a warning log and retraining suggestion if accuracy drops below 90%.
10. Write a final summary report to disk (performance_report.txt) and output the comprehensive monitoring results to the console.

Code:

1. IMPORT LIBRARIES

```
import os
import logging
import joblib
import pandas as pd
import numpy as np
from datetime import datetime
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
f1_score, classification_report, confusion_matrix)
```

```
print("=" * 60)
print("WEEK 11 - MODEL MONITORING")
print("Logging and Performance Metrics")
print("=" * 60)
```

2. CREATE MONITORING DIRECTORY

```
MONITORING_FOLDER = "week11_monitoring"
os.makedirs(MONITORING_FOLDER, exist_ok=True)
print("\n1. MONITORING FOLDER CREATED")
 print("-" * 60)
print(MONITORING_FOLDER)
```

3. CONFIGURE LOGGING

```
log_file = os.path.join(MONITORING_FOLDER, "model_monitoring.log")
logging.basicConfig(filename=log_file,level=logging.INFO,
format="%(asctime)s - %(levelname)s - %(message)s", force=True)
logging.info("Model monitoring started.")
```

4. LOAD MODEL

```
model_path = os.path.join("model_artifacts", "iris_model.pkl")
if os.path.exists(model_path):
    model = joblib.load(model_path)
    print("\n2. MODEL LOADED")
    print("-" * 60)
    print("Model:",model_path)
    logging.info("Model loaded successfully.")
else:
    print("Model not found.")

print("Please run Week 10 first.")
raise FileNotFoundError("iris_model.pkl not found. "
"Run Week 10 before Week 11.")
```

5. LOAD TEST DATA

```
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)
```

6. CREATE TEST SET

```
X_train, X_test, y_train, y_test = train_test_split(
X,y,test_size=0.20,random_state=42,stratify=y)
print("\n3. TEST DATA READY")
print("-" * 60)
print("Test samples:",len(X_test))
```

7. GENERATE PREDICTIONS

```
predictions = model.predict(X_test)

print("\n4. PREDICTIONS GENERATED")
print("-" * 60)
print(predictions)
logging.info(f"Generated {len(predictions)} predictions.")
```

8. CALCULATE PERFORMANCE METRICS

```
accuracy = accuracy_score(y_test, predictions)
precision = precision_score(y_test, predictions, average="weighted",
zero_division=0)
recall = recall_score(y_test,predictions,average="weighted",zero_division=0)
f1 = f1_score(y_test,predictions,average="weighted",zero_division=0)
```

9. DISPLAY METRICS

```
print("\n5. MODEL PERFORMANCE METRICS")
print("-" * 60)
print("Accuracy :",round(accuracy, 4))
print("Precision:",round(precision, 4))
print("Recall :",round(recall, 4))
print("F1 Score :",round(f1, 4))
```

10. LOG PERFORMANCE

```
f"Accuracy: {accuracy:.4f}")
logging.info(f"Precision: {precision:.4f}")
logging.info(f"Recall: {recall:.4f}")
logging.info(f"F1 Score: {f1:.4f}")
```

11. CONFUSION MATRIX

```
cm = confusion_matrix(y_test, predictions)
print("\n6. CONFUSION MATRIX")
print("-" * 60)
print(cm)
```

12. CLASSIFICATION REPORT

```
print("\n7. CLASSIFICATION REPORT")
print("-" * 60)
print(classification_report(y_test, predictions, target_names=iris.target_names))
```

13. CREATE MONITORING DATAFRAME

```
monitoring_data = pd.DataFrame({
"Timestamp": [datetime.now().strftime("%Y-%m-%d %H:%M:%S")],
"Accuracy": [accuracy],
```

```
"Precision": [precision],
"Recall": [recall],
"F1_Score": [f1]
})
print("\n8. MONITORING DATA")
print("-" * 60)
print(monitoring_data)
```

14. SAVE MONITORING CSV

```
monitoring_csv = os.path.join(MONITORING_FOLDER,
"performance_metrics.csv")
monitoring_data.to_csv(monitoring_csv, index=False)
print("Performance metrics saved:")
print(monitoring_csv)
```

15. PERFORMANCE ALERT

```
minimum_accuracy = 0.90
print("\n9. PERFORMANCE ALERT")
print("-" * 60)
if accuracy < minimum_accuracy:
    print(" WARNING: Model performance is low!")
    print("Recommended action: Retrain model.")
    logging.warning("Model accuracy is below threshold.")
else:
    print(" Model performance is GOOD.")
print("No retraining required.")
logging.info("Model performance is good.")
```

16. CREATE PERFORMANCE REPORT

```
report_file = os.path.join(MONITORING_FOLDER,"performance_report.txt")
with open(report_file,"w") as file:
    file.write("MODEL MONITORING REPORT\n")
    file.write("=" * 40 + "\n")
    file.write(f"Timestamp: {datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}\n")
    file.write(f"Accuracy: {accuracy:.4f}\n")
    file.write(f"Precision: {precision:.4f}\n")
    file.write(f"Recall: {recall:.4f}\n")
    file.write(f"F1 Score: {f1:.4f}\n")
if accuracy >= minimum_accuracy:
    file.write("Status: GOOD\n")
else:
    file.write("Status: WARNING\n")
```

```
17. FINAL STATUS
print("\n")
print("=" * 60)
print("MODEL MONITORING RESULT")
print("=" * 60)
print("Accuracy :",round(accuracy, 4))
print("Precision:",round(precision, 4))
print("Recall :",round(recall, 4))
print("F1 Score :",round(f1, 4))
print("-" * 60)
if accuracy >= minimum_accuracy:
   print("STATUS:MODEL PERFORMANCE GOOD")
else:
   print("STATUS:PERFORMANCE WARNING")
print("-" * 60)
print("Generated files:")
print("1.", log_file)
print("2.", monitoring_csv)
print("3.", report_file)
print("=" * 60)
print("\nWeek 11 exercise completed successfully!")
```

Output:

1. MONITORING FOLDER CREATED

week11_monitoring

2. MODEL LOADED

Model: model_artifacts/iris_model.pkl

3. TEST DATA READY

Test samples: 30

4. PREDICTIONS GENERATED

[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 1 0 2 1 1 2 2 1 0 2 0]

5. MODEL PERFORMANCE METRICS

Accuracy : 0.9
Precision: 0.9024
Recall : 0.9
F1 Score : 0.8997

6. CONFUSION MATRIX

[[10 0 0]
[0 9 1]
[0 2 8]]

7. CLASSIFICATION REPORT

precision recall f1-score support
setosa 1.00 1.00 1.00 10
versicolor 0.82 0.90 0.86 10
virginica 0.89 0.80 0.84 10
accuracy 0.90 30
macro avg 0.90 0.90 0.90 30
weighted avg 0.90 0.90 0.90 30

8. MONITORING DATA

Timestamp Accuracy Precision Recall F1_Score
0 2026-08-31 16:33:05 0.9 0.902357 0.9 0.899749
Performance metrics saved:
week11_monitoring/performance_metrics.csv

9. PERFORMANCE ALERT

Model performance is GOOD.
No retraining required.

MODEL MONITORING RESULT

Accuracy : 0.9
Precision: 0.9024
Recall : 0.9
F1 Score : 0.8997

STATUS:MODEL PERFORMANCE GOOD
Generated files:
1. week11_monitoring/model_monitoring.log
2. week11_monitoring/performance_metrics.csv
3. week11_monitoring/performance_report.txt

Week 11 exercise completed successfully!

Result:

Thus, the MLOPs program was successfully executed, and the expected output was obtained.

12. Model Monitoring (Part 2) - Drift Detection (Introduction) & Dashboard/Alerting

Aim:

To implement advanced model monitoring capabilities by simulating and detecting data drift, configuring automated operational alerts, and generating comprehensive visual dashboards using the evidently and matplotlib libraries.

Procedure:

1. Initialize a dedicated output directory (week12_monitoring) to safely store monitoring reports, dashboards, and logged data.
2. Load the reference dataset (Iris) representing the stable data distribution utilized during model training.
3. Clone the reference data to act as the current production dataset, and deliberately inject artificial mathematical drift into specific features (sepal length and petal length).
4. Calculate the absolute mean change between the reference and current datasets for all available features.
5. Implement manual drift detection by establishing a numerical threshold (0.5); flag any feature exceeding this threshold and systematically log the results.
6. Aggregate the monitoring results into a structured Pandas DataFrame and save them as a CSV for historical auditing.
7. Trigger automated system alerts via the Python logging module if any feature breaches the drift threshold.
8. Leverage the Evidently library to auto-generate a comprehensive, interactive HTML Data Drift dashboard.
9. Create and save a Matplotlib bar chart visually depicting the absolute mean change of each feature relative to the critical threshold line.
10. Output a conclusive system status report detailing the drift percentage and recommended operational actions.

Code:

```
import os
import logging
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from evidently import Report
from evidently.presets import DataDriftPreset
```

```
OUTPUT_FOLDER = "week12_monitoring"
os.makedirs(OUTPUT_FOLDER, exist_ok=True)
iris = load_iris()

reference_data = pd.DataFrame(iris.data, columns=iris.feature_names)
current_data = reference_data.copy()

current_data["sepal length (cm)"] = current_data["sepal length (cm)"] + 1.5
current_data["petal length (cm)"] = current_data["petal length (cm)"] + 1.0

reference_mean = reference_data.mean()
current_mean = current_data.mean()
mean_change = abs(current_mean - reference_mean)

threshold = 0.5
monitoring_results = []

for feature in reference_data.columns:
    change = mean_change[feature]
    status = "DRIFT DETECTED" if change > threshold else "NORMAL"

monitoring_results.append({
"Feature": feature,
"Reference Mean": reference_mean[feature],
"Current Mean": current_mean[feature],
"Mean Change": change,
"Threshold": threshold,
"Status": status
})

monitoring_df = pd.DataFrame(monitoring_results)
monitoring_df.to_csv(os.path.join(OUTPUT_FOLDER, "monitoring_results.csv"),
index=False)

drift_count = (monitoring_df["Status"] == "DRIFT DETECTED").sum()
total_features = len(reference_data.columns)

report = Report(metrics=[DataDriftPreset()])
result = report.run(reference_data=reference_data, current_data=current_data)
result.save_html(os.path.join(OUTPUT_FOLDER, "drift_report.html"))
```

```
plt.figure(figsize=(10, 5))
plt.bar(monitoring_df["Feature"], monitoring_df["Mean Change"])
plt.axhline(y=threshold, linestyle="--", color="red", label="Drift Threshold")
plt.xlabel("Features")
plt.ylabel("Absolute Mean Change")
plt.title("Feature Drift Monitoring")
plt.xticks(rotation=30, ha="right")
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_FOLDER, "drift_chart.png"), dpi=300)
```

Output:

1. REFERENCE DATA LOADED

Dataset shape: (150, 4)

2. CURRENT DATA CREATED

Current data shape: (150, 4)

3. ARTIFICIAL DATA DRIFT INTRODUCED

Drift introduced in:
- sepal length (cm)
- petal length (cm)

6. MEAN CHANGE

Reference Mean Current Mean Absolute Change
sepal length (cm) 5.843333 7.343333 1.500000e+00
sepal width (cm) 3.057333 3.057333 4.440892e-16
petal length (cm) 3.758000 4.758000 1.000000e+00
petal width (cm) 1.199333 1.199333 4.440892e-16

7. DRIFT DETECTION

sepal length (cm): DRIFT DETECTED (Change = 1.5000)
sepal width (cm): NORMAL (Change = 0.0000)
petal length (cm): DRIFT DETECTED (Change = 1.0000)
petal width (cm): NORMAL (Change = 0.0000)

8. MONITORING SUMMARY

Feature Reference Mean Current Mean Mean Change
Threshold Status
0 sepal length (cm) 5.843333 7.343333 1.500000e+00 0.5
DRIFT DETECTED
1 sepal width (cm) 3.057333 3.057333 4.440892e-16
0.5 NORMAL
2 petal length (cm) 3.758000 4.758000 1.000000e+00 0.5
DRIFT DETECTED

3 petal width (cm) 1.199333 1.199333 4.440892e-16
0.5 NORMAL

9. DRIFT SUMMARY
Total features: 4
Drifted features: 2
Drift percentage: 50.0 %
Alert Message: WARNING: DATA DRIFT DETECTED! Model performance should be
investigated.

FINAL MODEL MONITORING STATUS
STATUS: DRIFT DETECTED
ACTION: Investigate production data and consider model retraining.

Week 12 exercise completed successfully!

Result:

Thus, the MLOPs program was successfully executed, and the expected output was obtained.